Testing

Contents

Testing **1**
- Scope . . . 1
- Official requirements . . . 1
- Project policy . . . 1
- Checklist . . . 2
- References . . . 2
- Related . . . 2

Testing

Scope

What is tested, at which layer, and what must be green before merge.

Official requirements

Subject V.8 requires the team to *"set up specific one-off tests for each layer"* alongside agility and continuous integration. Per-layer testing is explicitly graded.

Project policy

Layers and what they prove

Layer	Tool	Proves	Uses a database
Domain unit	xUnit	invariants and rules in isolation	no
Application unit	xUnit + NSubstitute	a handler orchestrates correctly	no, repositories faked
Validation	xUnit	every command rejects bad input	no
Persistence integration	xUnit + Testcontainers	mappings, constraints, real SQL	yes, real engine in a container
API integration	`WebApplicationFactory`	routing, status codes, auth, error shape	yes
Realtime	SignalR test client	join authorisation, event delivery, reconnect	yes
Load	see `../load-testing/STANDARD.md`	capacity	yes

In-memory EF providers are **not** used for persistence tests. They do not enforce unique indexes or concurrency tokens, which are exactly the things we need proven.

Naming and shape

- Test project per production project: `<Project>.Tests`.
- Test name: `Method_Scenario_ExpectedResult` — `CastVote_WhenUserAlreadyVoted_ReturnsConflict`.
- Arrange / Act / Assert, visibly separated. One behaviour per test.
- No shared mutable state between tests. Tests run in any order.
- No `Thread.Sleep`. Wait on a condition with a timeout.
- **Never prove a negative by waiting.** "No message arrived" asserted with a timeout is slow and only probably true, and it inverts into a false failure the moment a message the test did not account for lands inside the window — see `L-042`. Use a **fence**: perform the action that must produce the thing, after the action that must not, and assert the first thing observed is the fence's. Where no fence exists, use a round trip over the same transport as a barrier: once it returns, whatever was already sent has arrived.

Mandatory coverage

These must have tests before the related feature is considered done:

- One vote per user per track under **concurrent** requests.
- Simultaneous playlist reorder by two users converges to one consistent order.
- A private event or playlist is invisible to a non-invited user — including over SignalR.
- A user cannot read or modify another user's data on any endpoint.
- Password reset invalidates existing sessions.
- Missing client context headers reject a mutating request.

Concurrency tests run the operation in parallel and assert the final database state. A sequential test proves nothing about a race.

Gate

- `dotnet test` green is required to merge. A failing or skipped test blocks the pull request.
- Coverage is measured and reported, but a percentage is not the gate; the mandatory list above is.

Checklist

- ☐ Every layer has tests
- ☐ Persistence tests run against a real engine in a container
- ☐ Concurrency cases tested in parallel, not sequentially
- ☐ Authorisation negative cases tested per endpoint and per hub
- ☐ No skipped tests on `main`

References

- xUnit — https://xunit.net/
- Testcontainers for .NET — https://dotnet.testcontainers.org/
- Integration tests in ASP.NET Core — https://learn.microsoft.com/aspnet/core/test/integration-tests

Related

`../../10-backend/ef-core-persistence/STANDARD.md` · `../load-testing/STANDARD.md` · `../../70-delivery/ci-cd-`